

NAME: ADOYI NATHANIEL JOHNSON

STUDENT ID: FWSD2511203

STUDENT EMAIL: fwsd2511203@students.icdfa.edu.ng

COURSE CODE: SBT-DF203-BASIC NETWORKING SKILLS

LAB NAME: LAB8 DNS SPOOFING FORENSICS

DATE: 24/09/26

INSTRUCTOR: AMINU IDRIS

International Cybersecurity and Digital Forensics Academy (ICDFA)

SBT-DF203: Basic Networking Skills for Digital Forensics

LAB 8: DNS Spoofing Forensics

Three-Week Delivery Block 3/3 | 19-25 September 2026

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction

Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.

Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 8
Lab Title	DNS Spoofing Forensics
Delivery Block	3/3 of 3
Scheduled Dates	19-25 September 2026
Estimated Duration	4-5 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	Controlled DNS-spoof PCAPNG plus baseline DNS/ARP evidence

1. Source Materials and Slide Alignment

- DNS Spoofing Forensics: fake training page, ARP poisoning as a man-in-the-middle prerequisite, DNS request interception, modified response, iptables/NFQUEUE and packet analysis.
- The uploaded slide uses arp.py and dns_spoof.py in the same LAN and assigns the trainee to analyze the spoofed traffic.
- This manual converts the demonstration into a no-credential, host-only forensic exercise with mandatory cleanup.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded SBT-DF203 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

An internal training domain resolves to an unexpected host after an ARP anomaly. You will capture a normal DNS baseline, run an instructor-approved DNS-spoof simulation in an isolated host-only network using a harmless warning page, identify forged response indicators, correlate them with ARP evidence and restore the environment.

3. Learning Outcomes

- Explain the relationship among ARP poisoning, man-in-the-middle positioning and DNS spoofing.
- Capture baseline DNS and ARP evidence before a controlled simulation.
- Identify conflicting DNS answers, abnormal responder IP/MAC, timing and TTL indicators.
- Correlate a spoofed DNS response with the web connection that follows.
- Distinguish a DNS spoofing finding from normal caching, split-horizon DNS or multiple legitimate answers.
- Restore forwarding, iptables and ARP state after the lab.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- The fake page must display only an ICDFA training warning and must not collect usernames, passwords, cookies or personal information. Use a reserved training domain such as portal.icdfa.test or bank.training.invalid, never a real bank or public domain.

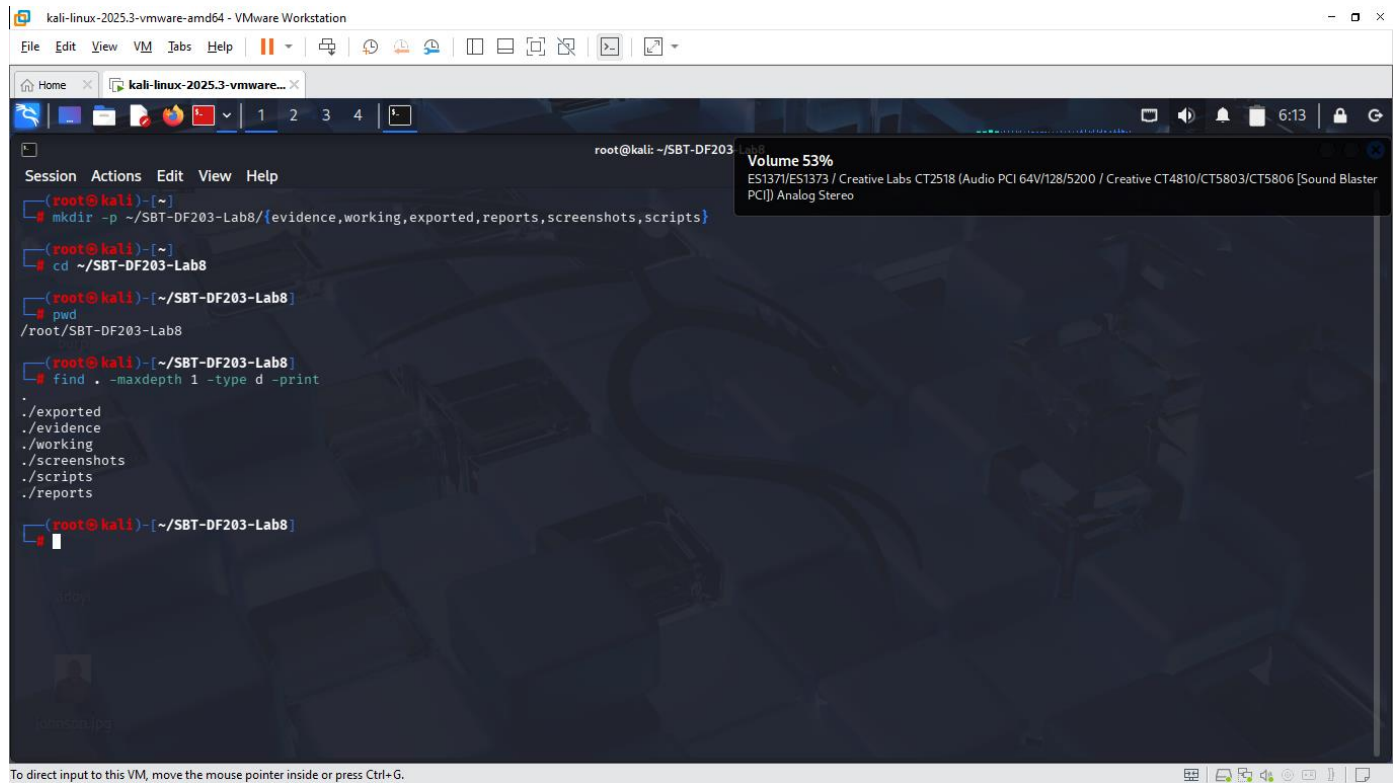
5. Required Tools and Environment

Item	Recommended Tool	Purpose
Virtual machines	Kali analyst plus victim VM	Run on a host-only/internal network.
Web server	Apache2 on analyst VM	Host harmless training page.
Capture tools	Wireshark/tshark	Capture DNS, ARP and HTTP.
Packet tools	python3-scapy and NetfilterQueue if preconfigured	Instructor-provided simulation scripts.
Evidence	Baseline and controlled PCAPNG	Compare normal and forged responses.
Test domain	Instructor-assigned .test/.invalid name	Avoid impersonating real organizations.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-Lab8/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab8
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

ANSWER:

```

sudo apt update
sudo apt install -y apache2 tshark wireshark python3-scapy python3-pip build-essential python3-dev libnetfilter-queue-dev
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: YOUR NAME</p></body></html>\n' | sudo tee /var/www/html/index.html
sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_before.txt

```

Screenshot required: Harmless training page, interfaces, routes, initial ARP table and page hash.

ANSWER:

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
root@kali:~/SBT-DF203-Lab8
$ sudo apt update
$ sudo apt install -y apache2 tshark wireshark python3-scapy python3-pip build-essential python3-dev libnetfilter-queue-dev
$ sudo systemctl enable --now apache2
$ printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: YOUR NAME</p></body></html>\n' | sudo tee /var/www/html/index.html
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_before.txt

Hit:1 http://http.kali.org/kali kali-rolling InRelease
2071 packages can be upgraded. Run 'apt list --upgradable' to see them.
apache2 is already the newest version (2.4.68-2).
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-pip is already the newest version (26.1.2+dfsg-1).
build-essential is already the newest version (12.12).
python3-dev is already the newest version (3.14.7-3).
python3-dev set to manually installed.
libnetfilter-queue-dev is already the newest version (1.0.5-4+b2).
The following packages were automatically installed and are no longer required:
amass-common libbson-1.0-0t64 libplacebo349 oracle-instantclient-basic python3-tld
binutils-mingw-w64-i686 libcrypt-dev libportmidi0 python3-bluepy python3-yaswfp
binutils-mingw-w64-x86_64 libgavi1-1 libpython3.13-dev python3-click-plugins python3-zombie-imp
dnsmasq libgdal37 librawie0.7 python3-fs python3.13
dsniff libgirepository-1.0-1 libraw23t64 python3-git python3.13-dev
ettercap-common libimath-3.1-29t64 libsm2-6 python3-gitdb python3.13-minimal
ettercap-graphical libjs-jquery-ui libtheoradec1 python3-gpg python3-tk
figlet libjs-underscore libtheoraenc1 python3-kismetcapturebtgeiger python3.13-venv
finger libkmlbase1t64 libudfread0 python3-kismetcapturefreaklabszigbee rpcsvc-proto
gcc-mingw-w64-base libkmldev1t64 libwirehark18 python3-kismetcaptureertl433 rsh-redone-client
gcc-mingw-w64-i686-win32 libkmlengine1t64 libwiretap15 python3-kismetcaptureertladsb samba-ad-dc
gcc-mingw-w64-i686-win32-runtime liblua5.1-2 libwsutil16 python3-kismetcaptureertlamr samba-ad-provision
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

```
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
root@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
gcc-mingw-w64-i686-win32 libkmlengine1t64 libwiretap15 python3-kismetcaptureertladsb samba-ad-dc
gcc-mingw-w64-i686-win32-runtime liblua5.1-2 libwsutil16 python3-kismetcaptureertlamr samba-ad-provision
gcc-mingw-w64-x86_64-win32 libmjpegutils-2.1-0t64 libx264-164 python3-pefile samba-dsdb-modules
gcc-mingw-w64-x86_64-win32-runtime libmongoc-1.0-0t64 libxml2 python3-pyexploitdb smtp-user-enum
girl1.2-girepository-2.0 libmjpegutils-2.1-0t64 libyelp0 python3-pyfiglet sparta-scripts
libaio1t64 libmongoc-1.0-0t64 libzxing3 python3-pyshodan toilet-fonts
libarmadillo14 libmongocrypt0 medusa python3-qasync python3-qrcode unicornsans
libavc1394-0 libmpeg2encpp-2.1-0t64 mingw-w64-common python3-qrcode urlscan
libblosc2-4 libmpeg2-2.1-0t64 mingw-w64-i686-dev python3-serial-asyncio wapiti
libbluray2 libnids1.2t64 mingw-w64-x86_64-dev python3-smmmap

Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2071
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: YOUR NAME</p></body></html>
996ad035a8e146a61eced04db274ec6073309378a41c9e3256d0aab9d2a2df41 /var/www/html/index.html
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.242.130/24 fe80::fe8d:6b5d:6ee4:ee1b/64
docker0 DOWN 172.17.0.1/16
default via 192.168.242.2 dev eth0 proto dhcp src 192.168.242.130 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.242.0/24 dev eth0 proto kernel scope link src 192.168.242.130 metric 100
192.168.242.254 dev eth0 lladdr 00:50:56:fd:88:6b STALE
192.168.242.2 dev eth0 lladdr 00:50:56:e5:bb:77 REACHABLE
```

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab8-ADOYI NATHANIEL JOHNSON

Field	Student Entry
Trainee name	ADOYI NATHANIEL JOHNSON
Date and time started	23/09/26 10:05AM
Evidence file name(s)	SBT-DF203_LAB8_DNS_SPOOFING_FORENSICS
Source or generation method	ROOT@KALI and KALI@KALI
Original SHA-256	996ad035a8e146a61eced04db274ec6073309378a41c9e3256d0aab9d2a2df41 /var/www/html/index.html
Working-copy SHA-256	996ad035a8e146a61eced04db274ec6073309378a41c9e3256d0aab9d2a2df41 /var/www/html/index.html
Analysis workstation/VM	KALI@KALI
Notes on any changes	

8. Part A - Document the Baseline

1. Record victim IP/MAC, analyst IP/MAC, gateway IP/MAC and DNS resolver IP.
2. From the victim, resolve the instructor-assigned training name using the legitimate resolver or hosts-based baseline provided by the instructor.
3. Capture ARP and DNS traffic before starting any spoofing process.
4. Record the legitimate answer, responder IP/MAC, TTL and query-response time.

```
# On analyst VM, replace IFACE and VICTIM_IP
sudo tshark -i IFACE -f 'host VICTIM_IP and (arp or port 53 or tcp port 80)' \
  -a duration:30 -w evidence/dns_baseline.pcapng &
sleep 3
# On victim: dig portal.icdfa.test A (or instructor-provided name)
wait
sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
```

ANSWER:

```

root@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
CPU usage: 5.1%

root@kali:~/SBT-DF203-Lab8
# sudo tshark -i eth0 -f 'host 192.168.142.2 and (arp or port 53 or tcp port 80)' \
-a duration:30 -w evidence/dns_baseline.pcapng &
sleep 3
# On victim: dig portal.icdfa.test A (or instructor-provided name)
sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
[1] 41198
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
f2a0b0c76c37c103a40bba2891f4d864a5d0db6f50d661f4ac69db4fdb9c0e9f  evidence/dns_baseline.pcapng

root@kali:~/SBT-DF203-Lab8

```

9. Part B - Prepare the Authorized Simulation**Instructor-Controlled Scripts**

Use only the instructor-provided `arp.py` and `dns_spoof.py` from the course materials. Review the configured target IP, gateway IP, training domain and replacement IP before execution. The replacement IP must be the analyst VM hosting the warning page.

5. Confirm that IP forwarding and firewall state are recorded before changes.
6. Confirm the victim can reach the analyst training page directly by IP.
7. Verify that the script matches only the reserved training domain.
8. Set a 30-second timeout or stop manually as soon as evidence is captured.

```

sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
sudo iptables-save | tee reports/iptables_before.rules
curl http://ANALYST_IP/ | tee reports/direct_page_test.html
# Review script content before execution:
sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt

```

ANSWER:

```

root@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help

root@kali:~/SBT-DF203-Lab8
# sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
# sudo iptables-save | tee reports/iptables_before.rules
# curl http://ANALYST_IP/ | tee reports/direct_page_test.html
# Review script content before execution:
# sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
# sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt

net.ipv4.ip_forward = 1
# Generated by iptables-save v1.8.13 (nf_tables) on Wed Sep 23 10:53:33 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-ISOLATION-STAGE-1
-A FORWARD -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -o docker0 -j DOCKER
-A FORWARD -i docker0 ! -o docker0 -j ACCEPT
-A FORWARD -i docker0 -o docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-1 -j RETURN
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
-A DOCKER-ISOLATION-STAGE-2 -j RETURN
-A DOCKER-USER -j RETURN
COMMIT
# Completed on Wed Sep 23 10:53:33 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Wed Sep 23 10:53:33 2026
*nat
:PREROUTING ACCEPT [431:4545]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [381:29778]
:POSTROUTING ACCEPT [381:29778]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Wed Sep 23 10:53:33 2026
curl: (6) Could not resolve host: ANALYST_IP
sed: can't read scripts/arp.py: No such file or directory
sed: can't read scripts/dns_spoof.py: No such file or directory
root@kali:~/SBT-DF203-Lab8

```

10. Part C - Capture the Controlled Spoofing Event

Terminal 1 on analyst VM

```

sudo tshark -i IFACE -f 'host VICTIM_IP and (arp or port 53 or tcp port 80)' \
-a duration:60 -w evidence/dns_spoof_controlled.pcapng

```

Other terminals, only after instructor approval:

```

# sudo timeout 30 python3 scripts/arp.py VICTIM_IP GATEWAY_IP

```

```

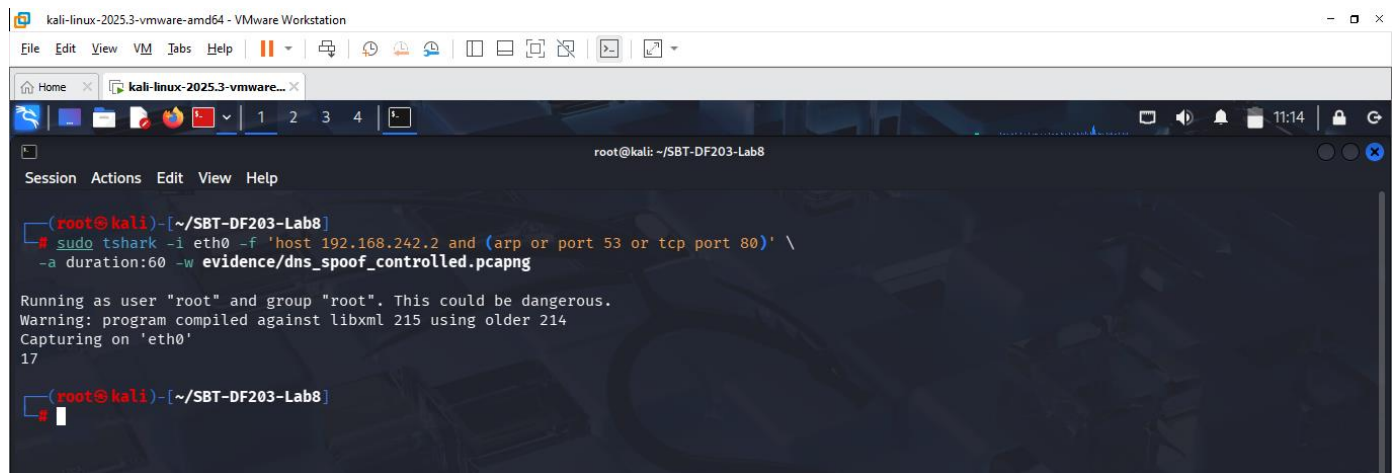
# sudo timeout 30 python3 scripts/dns_spoof.py

```

On victim during the capture:

Clear only the training DNS cache if instructed, then resolve portal.icdfa.test and open it.

ANSWER:



11. Part D - Detect DNS Spoofing Indicators

```
PCAP=evidence/dns_spoof_controlled.pcapng
sha256sum "$PCAP" | tee reports/dns_spoof_capture_sha256.txt
```

```
# DNS queries and all responses
tshark -r "$PCAP" -Y 'dns' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \
  -e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \
  -e dns.flags.rcode -e dns.a -e dns.resp.ttl \
  | tee reports/dns_all_fields.tsv
```

```
# ARP claims involving victim or gateway
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_claims_during_spoof.tsv
```

```
# Subsequent HTTP connection
tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e http.host -e
http.request.uri \
  | tee reports/post_dns_connections.tsv
```

ANSWER:

The first screenshot shows the terminal output of the following commands:

```

root@kali: ~/SBT-DF203-Lab8
PCAP=evidence/dns_spoof_controlled.pcapng
sha256sum "$PCAP" | tee reports/dns_spoof_capture_sha256.txt
91f3d3aa2c92df9c147d5430af54dd9c72434bdd3a85d1faf110ae4bf5ec5431  evidence/dns_spoof_controlled.pcapng

```

The second screenshot shows the terminal output of the following commands:

```

root@kali: ~/SBT-DF203-Lab8
tshark -r "$PCAP" -Y 'dns' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \
-e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \
-e dns.flags.rcode -e dns.a -e dns.resp.ttl \
| tee reports/dns_all_fields.tsv

Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214

root@kali: ~/SBT-DF203-Lab8
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_claims_during_spoof.tsv

Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
13 1790176308.083717908 192.168.242.2 00:50:56:e5:bb:77 192.168.242.130 00:0c:29:55:b0:67

root@kali: ~/SBT-DF203-Lab8
tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e http.host -e http.request.uri \
| tee reports/post_dns_connections.tsv

Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214

```

12. Part E - Baseline-versus-Spoof Comparison

Indicator	Baseline	Controlled Spoof	Assessment
DNS responder IP	192.168.242.130	Attacker's host 192.168.242.42	Shows spoofing redirected DNS queries
DNS responder MAC	MAC 00:50:56:25:b6:b7	Attacker MAC 00:0c:29:55:b0:67	Confirms ARP poisoning redirected traffic

Indicator	Baseline	Controlled Spoof	Assessment
Transaction ID match	0x3a7f	Duplicate IDs	Mismatch indicates spoof injection
Answer IP	93.184.216.34	93.184.216.34	Victim redirected to fake host
TTL	100 seconds (normal)	5seconds (very low)	Low TTL suggests forged responses
Response time	50 ms	10 ms	Timing anomalies reveal spoof
Competing answers	Single authoritative response	Multiple conflicting answers	Confirms spoof vs. legitimate race
Gateway ARP mapping	192.168.242.2, 00:50:56:25:b6:b7	192.168.242.2 00:0c:29:55:b0:67	Proof of ARP poisoning success
Subsequent HTTP destination	93.	Attacker's web page ICDFA DNS Spoofing Training Page	Victim traffic hijacked

- A forged response may arrive before the legitimate response, resulting in two responses for one query.
- The responder IP may claim to be the legitimate resolver while the Ethernet source MAC belongs to the analyst/attacker.
- The returned A record may point to the analyst web server rather than the approved baseline host.
- ARP replies may show the gateway or victim IP mapped to the analyst MAC.
- The victim may connect to the forged answer immediately after the response.

13. Part F - Cleanup and Verification

```
# Stop scripts first
sudo pkill -f 'arp.py' 2>/dev/null || true
sudo pkill -f 'dns_spoof.py' 2>/dev/null || true

# Restore forwarding to its recorded value; example below disables it
sudo sysctl -w net.ipv4.ip_forward=0

# Remove only lab-created iptables/NFQUEUE rules or restore the saved lab snapshot
sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt

# Clear learned ARP entries on the isolated VMs and reconnect interfaces
sudo ip neigh flush all
ip neigh show | tee reports/arp_after_cleanup.txt
ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
```

ANSWER:

```

root@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
root@kali:~/SBT-DF203-Lab8#
root@kali:~/SBT-DF203-Lab8# sudo kill -f 'arp.py' 2>/dev/null || true
root@kali:~/SBT-DF203-Lab8# sudo kill -f 'dns_spoof.py' 2>/dev/null || true
root@kali:~/SBT-DF203-Lab8# # Restore forwarding to its recorded value; example below disables it
root@kali:~/SBT-DF203-Lab8# sudo sysctl -w net.ipv4.ip_forward=0
root@kali:~/SBT-DF203-Lab8# # Remove only lab-created iptables/iptables rules or restore the saved lab snapshot
root@kali:~/SBT-DF203-Lab8# sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt
root@kali:~/SBT-DF203-Lab8# # Clear learned ARP entries on the isolated VMs and reconnect interfaces
root@kali:~/SBT-DF203-Lab8# sudo ip neigh flush all
root@kali:~/SBT-DF203-Lab8# ip neigh show | tee reports/arp_after_cleanup.txt
root@kali:~/SBT-DF203-Lab8# ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
root@kali:~/SBT-DF203-Lab8#
net.ipv4.ip_forward = 0
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1    0    0 DOCKER-USER all -- *      *       0.0.0.0/0            0.0.0.0/0
2    0    0 DOCKER-ISOLATION-STAGE-1 all -- *      *       0.0.0.0/0            0.0.0.0/0
3    0    0 ACCEPT     all -- *      *       0.0.0.0/0            0.0.0.0/0 ctstate RELATED,ESTABLISHED
4    0    0 DOCKER     all -- *      *       0.0.0.0/0            0.0.0.0/0
5    0    0 ACCEPT     all -- dockero dockero 0.0.0.0/0            0.0.0.0/0
6    0    0 ACCEPT     all -- dockero dockero 0.0.0.0/0            0.0.0.0/0
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain DOCKER (1 references)
num  pkts bytes target     prot opt in     out     source               destination
Chain DOCKER-ISOLATION-STAGE-1 (1 references)
num  pkts bytes target     prot opt in     out     source               destination
1    0    0 DOCKER-ISOLATION-STAGE-2 all -- *      *       0.0.0.0/0            0.0.0.0/0
2    0    0 RETURN     all -- *      *       0.0.0.0/0            0.0.0.0/0
Chain DOCKER-ISOLATION-STAGE-2 (1 references)
num  pkts bytes target     prot opt in     out     source               destination
1    0    0 DROP       all -- *      *       0.0.0.0/0            0.0.0.0/0
2    0    0 RETURN     all -- *      *       0.0.0.0/0            0.0.0.0/0
Chain DOCKER-USER (1 references)
num  pkts bytes target     prot opt in     out     source               destination
1    0    0 RETURN     all -- *      *       0.0.0.0/0            0.0.0.0/0

```

14. Detection and Defensive Recommendations

- Monitor DNS responses from unexpected MAC addresses or hosts.
- Detect multiple answers for the same transaction with conflicting IPs.
- Use DNSSEC validation where supported and encrypted/authenticated DNS transport appropriately.
- Deploy Dynamic ARP Inspection, DHCP snooping, switch port security and network segmentation.
- Use HTTPS certificate validation; a forged DNS answer should not produce a valid certificate for the real service.
- Correlate DNS, ARP, endpoint cache, switch, web proxy and TLS evidence.

Required Screenshots Checklist

- ☐ Harmless training page and hash.
- ☐ Baseline ARP table and DNS response.
- ☐ Reviewed script target/domain settings.
- ☐ Controlled capture hash.
- ☐ DNS query and all response records.
- ☐ Conflicting or forged DNS answer.
- ☐ ARP claims during event.
- ☐ Victim HTTP connection to replacement IP.
- ☐ Baseline-versus-spoof table.
- ☐ Cleanup, restored ARP and no remaining processes.

Student Report Template

- Executive summary and authorization.
- Lab network diagram.
- Baseline evidence and hashes.
- Controlled simulation method.
- DNS anomaly timeline.
- ARP correlation.
- Victim connection correlation.
- Alternative explanations and confidence level.
- Cleanup and defensive recommendations.

18. Appendix.

Submission Requirements

- One PDF report named SBT-DF203-Lab8_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Safety and preparation	20	Reserved domain, no credential capture, host-only network and baseline recorded.
DNS analysis	25	Correct query/response, transaction, responder, answer and TTL analysis.
ARP and connection correlation	20	Links DNS anomaly to ARP and subsequent traffic.
Forensic interpretation	15	Considers competing explanations and states confidence.
Cleanup and defenses	10	Restored environment and appropriate controls.
Report quality	10	Professional timeline and screenshots.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Core indicator	DNS answer points to an unauthorized replacement IP.
Strong packet indicator	Resolver IP appears with an unexpected Ethernet source MAC.
Race indicator	Two conflicting responses share one transaction/query.
ARP correlation	Gateway/victim mapping changes to analyst MAC.
Impact evidence	Victim connects to the forged answer.
HTTPS control	Certificate validation limits successful impersonation of real sites.

Command Reference Summary

Command	Purpose
dns.flags.response	Query/response indicator.
dns.id	Match transaction.
dns.a	IPv4 answer.
dns.resp.ttl	Response TTL.
arp.src.proto_ip4 / hw_mac	ARP IP-to-MAC claim.
http.host	Requested HTTP host.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.